Quantum Wave Functions in 1D Systems

Ben Holmes

December 2025

Contents

1 Time Independent One-Dimensional Well

1.1 General Equations

Imagine a particle inside an infinitely deep well. Inside this well there is no potential energy. Using the time independent Schrodinger equation:

$$-\frac{\hbar^2}{2m}\frac{d^2\psi}{dx^2} + V(x)\psi(x) = E\psi(x)$$

we can formulate a general Wave Function;

As stated before, there is no potential energy in this well therefore we can rewrite the equation as:

$$-\frac{\hbar^2}{2m}\frac{d^2\psi}{dx^2} = E\psi(x)$$

The general solution to this is given by

$$\psi(x) = Asin(kx) + Bcos(kx)$$

Our boundary conditions state that

$$\psi(0) = 0 \text{ and } \psi(L) = 0$$

so if

$$\psi(x) = Asin(k0) + Bcos(k0)$$

$$B = 0$$

This means we can eliminate the cosine function from our Wave Function completely.

1.2 Solving for *k*

$$\frac{d\psi}{dx} = kAcos(kx)$$

$$\frac{d^2\psi}{dx^2} = -k^2Asin(kx)$$

Since

$$\psi(x) = Asin(kx)$$

$$\frac{d^2\psi}{dx^2} = -k^2\psi(x)$$

Comparing this to the original Schrodinger equation we find k to be:

$$k = \sqrt{\frac{2mE}{\hbar^2}}$$

which can also be written as:

$$k = \sqrt{\frac{8\pi^2 mE}{h^2}}$$

Plugging k back into our Wave Function

$$\psi(x) = A \sin\left(\sqrt{\frac{8\pi^2 mE}{h^2}}\, x\right)$$

1.3 Solving for *A*

Recalling our boundary conditions $x = L = 0$

$$0 = Asin(\frac{8\pi^2 mE}{h^2})^{\frac{1}{2}} L$$

Since $A \neq 0$ the only way for this to be true is when

$$(\frac{8\pi^2 mE}{h^2})^{\frac{1}{2}} L = n\pi$$

$$\psi = A \sin \frac{n\pi}{L} x$$

Now to Find A we must recall how the total probability of finding the particle in the well is 1 , as there is no way that the particle could escape. To get a probability distribution for this position we must do ψ^2. Knowing this we can say:

$$\int_0^L A^2 sin^2 \frac{n\pi x}{L} dx = 1$$

$$A = \sqrt{\frac{2}{L}}$$

Finally putting our wave function together, we can say that in our 1D infinite Well

$$\psi(x) = \sqrt{\frac{2}{L}} sin(\frac{n\pi x}{L})$$

1.4 Calculating Energy

By rearranging our existing formulas we can work out the the energy of the specific particles in our well in specific states

$$E = \frac{n^2 \pi^2 \hbar^2}{2mL^2}$$

For an electron confined in a well of width $L = 1\,\text{nm}$, the ground state energy is:

$$\frac{\pi^2 \times \hbar^2}{2 \times m_e \times L^2} = 6.025 \times 10^{-20} J$$

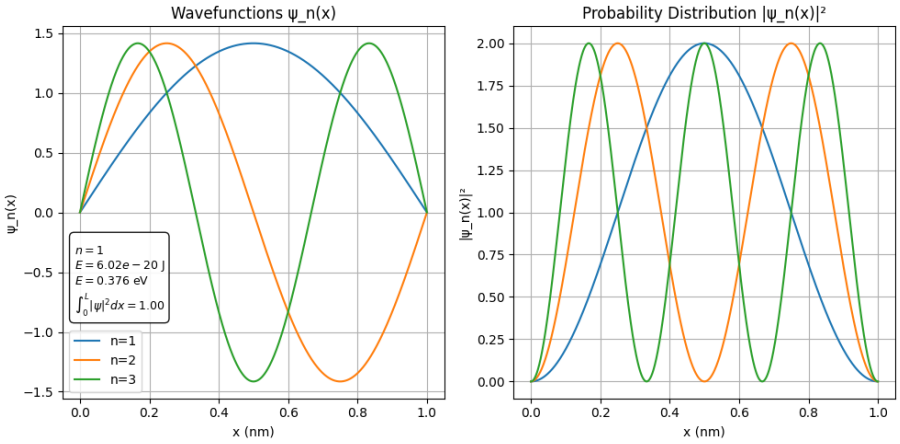

Figure 1: *Left*: Wavefunction of our particle *Right*: Probability of finding our particle at a specific location

1.5 Findings

From this One-Dimensional Potential Well model many important observations can be made. We can see that the probability of finding the wave at a point in the well is not uniform. It changes with position and is as defined 0 at the walls of the well. We can also see how the particle is most likely to be found where the Wave Function peaks. This means that as the energy level increases the number of peaks in the probability distribution also increases.

2 Time Dependent One-Dimensional Well

2.1 General Equations

Now that we have modeled a time-independent particle in a well, we can extend our analysis to the time-dependent case. For a particle in a stationary potential, our equation becomes

$$\Psi(x,t) = \psi(x)\, e^{-iE_n t/\hbar}$$

where $\psi(x)$ is the time-independent function found previously, and E_n is the corresponding energy value.

2.2 Simulation

Our graphing shows us how the real and imaginary parts oscillate individually in time however the probability distribution is constant. The probability distribution $|\Psi(x,t)|^2$ for a particle in a stationary state is therefore time-independent, which is a key feature of stationary states in quantum mechanics.

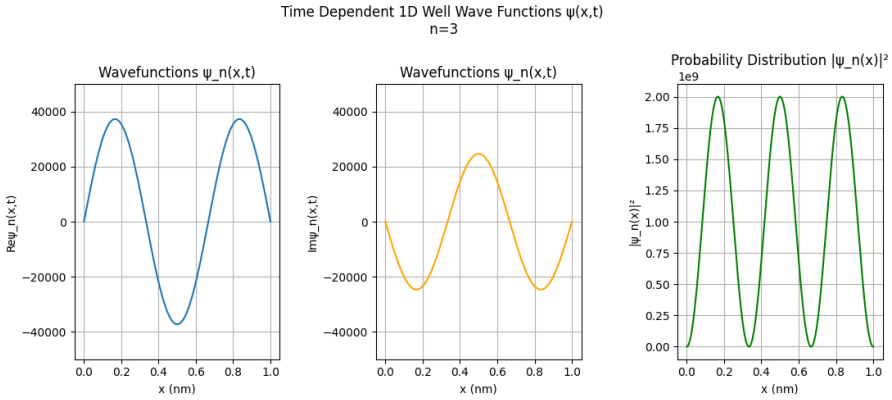

Figure 2: *Left*: Real part of our Wavefunction *Middle*:Imaginary part of our Wavefunction *Right*: Probability Distribution

3 Time Dependent Superpositions

Now we have a solid foundation in both the time dependent and time independent case we can start to think about superpositions. A superposition can be written as a linear combination of stationary states:

$$\Psi(x,t) = \sum_n C_n \psi_n(x) e^{-iE_n t/\hbar}$$

where C_n is the coefficient determining the contribution of each state, $\psi(x)$ are the stationary wavefunction described earlier and E_n is the corresponding energy value. Unlike the stationary states, the probability distribution for a superposition will now also oscillate in time. This happens because both states combined create interference patterns which means that at times the probability of finding the particle in a particular region of the well increases while at other times it decreases showing how our system is dynamic and constantly changing until observed.

In our model we chose the coefficient C_n to be equal for each state. We used $\frac{1}{\sqrt{2}}$ for each state. Knowing that $|C_n^2|$ gives us the probability of find the particle in that state we can say there is an equal chance $(\frac{1}{2})$ of finding the particle at each state. This also ensures the wave is normalized as

$$\int_0^L (|\psi_1(x) + \psi_2(x)|^2) dx = 2$$

We must normalize this to 1. Which is why our final wave function looks like:

$$\Psi(x,t) = \frac{1}{\sqrt{2}} (\psi_1(x)\, e^{-iE_1 t/\hbar} + \psi_2(x)\, e^{-iE_2 t/\hbar})$$

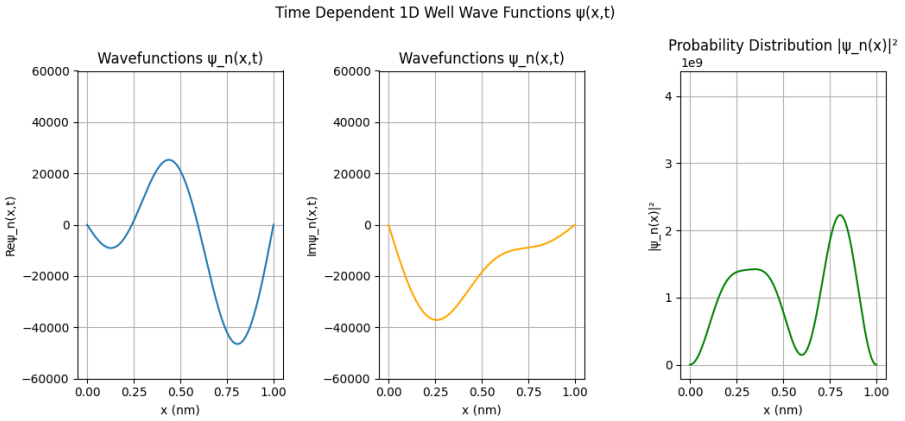

Figure 3: Wavefunction of a superposition of the first three energy states (n=1,2,3) of a quantum system.

3.1 Adding States

If we wanted to add more than just two possible states then our function once again no longer normalizes to 1. As a General Rule for equal probabilities C_n is:

$$\frac{1}{\sqrt{N}}$$

where N is the number of unique states

As expected our visualization now shows a more complex wave function with interference pattern from the 3 states. We can now also see how the probability distribution is time dependent.

4 Implementing Momentum Wave Function

4.1 Momentum Wave Equation

The momentum-space wave function $\phi(p,t)$ isn't as straightforward instead it is obtained as the Fourier transform of the position-space wave function $\psi(x,t)$. Since I don't yet understand the FFT I researched the general expression for converting position-space into momentum. For a one-dimensional system, this can be written as:

$$\phi(p,t) = \frac{1}{\sqrt{2\pi\hbar}} \int_0^L \psi(x,t)\, e^{-ipx/\hbar}\, dx$$

The corresponding momentum probability density is just the same as the position-space. We take the absolute value, then square it.

$$P(p) = |\phi(p,t)|^2$$

this results in a probability density which is time-independent for stationary states and interestingly remains constant in time even for superpositions in an infinite potential well. For a single stationary state, the momentum probability distribution is symmetric about $p = 0$. This doesn't mean the particle isn't moving it just means that the particle has no preferred direction of motion on average. This symmetry arises because the particle is equally likely to be moving to the left or to the right within the well.

However after implementing this into my code the heavy integration across a large amount of data meant my program had a huge bottle neck. To fix this I tried to gasp the very basics of the FFT in python in order to add a momentum space wavefunction. Once I got a seemly correct output the graphs had large FFT Artifacts so I had to take a new approach analytical approach.

4.2 Deriving a Momentum Space Formula

First of all we can express the Momentum Wavefunction as a Fourier transform of our previous Position-Space Function

$$\Phi(p) = \frac{1}{2\pi\hbar}\int_{-\infty}^{\infty} \psi(x) e^{\frac{-ipx}{\hbar}}\, dx$$

Subbing in $\psi(x)$ and removing constants we get

$$\Phi(p) = \sqrt{\frac{1}{\pi\hbar L}}\int_{0}^{L} sin(\frac{n\pi x}{L}) e^{\frac{-ipx}{\hbar}}\, dx$$

Next using Euler's Formula for sine:

$$sin(x) = \frac{e^{ix} - e^{-ix}}{2i}$$

we can say

$$sin(\frac{n\pi x}{L}) = \frac{e^{i\frac{n\pi x}{L}} - e^{-i\frac{n\pi x}{L}}}{2i}$$

Our Momentum Wave Function can now be written as:

$$\Phi(p) = \sqrt{\frac{1}{\pi\hbar L}} \times \frac{1}{2i}\int_{0}^{L} (e^{i\frac{n\pi x}{L}} - e^{-i\frac{n\pi x}{L}}) e^{\frac{-ipx}{\hbar}}\, dx$$

Expanding then simplifying we get:

$$\frac{1}{2i}\int_{0}^{L} (e^{ix(\frac{n\pi}{L} - \frac{p}{\hbar})} - e^{-ix(\frac{n\pi}{L} + \frac{p}{\hbar})}) dx$$

Using $\int e^{ikx} = \frac{e^{ikx}}{ik}$ we can evaluate from L to 0 which gives

$$\frac{e^{ikL}}{ik} - \frac{e^{ik0}}{ik} = \frac{e^{ikL} - 1}{ik}$$

Doing this for both K values we get

$$k_1 : \frac{e^{in\pi - \frac{ipL}{\hbar}} - 1}{i(\frac{n\pi}{L} - \frac{p}{\hbar})}$$

$$k_2 : \frac{1 + e^{-in\pi - \frac{ipL}{\hbar}}}{i(\frac{n\pi}{L} + \frac{p}{\hbar})}$$

knowing $e^{in\pi} = e^{-in\pi} = (-1)^n$ we can simplify to

$$k_1 : \frac{-1^n e^{-\frac{ipL}{\hbar}} - 1}{i(\frac{n\pi}{L} - \frac{p}{\hbar})}$$

$$k_2 : \frac{1 - (-1^n) e^{-\frac{ipL}{\hbar}} - 1}{i(\frac{n\pi}{L} + \frac{p}{\hbar})}$$

Putting this back into our integral and factoring

$$\frac{1}{2}[1 - (-1)^n e^{-\frac{ipL}{\hbar}}](\frac{1}{\frac{n\pi}{L} - \frac{p}{\hbar}} + \frac{1}{\frac{n\pi}{L} + \frac{p}{\hbar}})$$

the identity

$$\frac{1}{a-b} + \frac{1}{a+b} = \frac{2a}{a^2 - b^2}$$

can be used then remembering to multiply by the half leaves

$$\frac{a}{a^2 - b^2}$$

becoming

$$\Phi_n(p) = \frac{1}{\sqrt{\pi\hbar L}} \frac{\frac{n\pi}{L}[1 - (-1)^n e^{-\frac{ipL}{\hbar}}]}{(\frac{n\pi}{L})^2 - (\frac{p}{\hbar})^2}$$

5 Expectation Values

Expectation Values represent the average outcome of measuring a property of our particle, taking into account the effect of each superposition.

Table 1: $\langle x \rangle$for multiple states at $t = 0.0$

Highest state in superposition	$\langle x \rangle$(nm)
1	0.49999999999999994
2	0.3198734513025106
3	0.2502245191394815
4	0.20620992302234345
5	0.17755959052388465

5.1 Expectation Values of Position

The expectation values for position can expressed as

$$\langle x \rangle = \int_0^L x|\Psi(x,t)|^2 dx$$

Since $\langle x \rangle$ describes the average location of the particle if we measured its position many times in the same quantum system. For our particle in a 1nm well if we remove the effect of superposition we would expect to find the particle in the midpoint of the well - 0.5nm. The table shows this and also shows the effect of superpositions:

These results show how the expectation value of position shifts as more states are added to the superposition. For a single state, the particle is most likely found at the center of the well (0.5nm), but adding higher energy states introduces asymmetry in the probability distribution, pulling the average position away from the center. As more states are included, the contributions from different states interfere. These values oscillate in time and the system evolves, constructive and destructive interference causes the probability distribution to shift back and forth, making $\langle x \rangle$ a time-dependent quantity.

5.2 Uncertainty of Position

The uncertainty in position, Δx, measures how spread out a particle's position is in a given quantum state. It quantifies the range of locations where the particle is likely to be found and is given by the formula

$$\Delta x = \sqrt{\langle x^2 \rangle - \langle x \rangle^2}$$

5.3 Expectation Values of Momentum

The expectation value of momentum is expressed as

$$\langle p \rangle = \int_{-\infty}^{\infty} p \, |\phi(p,t)|^2 \, dp$$

5.4 Uncertainty of Momentum

and the momentum uncertainty is given by

$$\Delta p = \sqrt{\langle p^2 \rangle - \langle p \rangle^2}$$

This quantity measures how widely spread the momentum values are from the mean.

6 Heisenberg Uncertainty Principle

Heisenberg's Uncertainty Principle is a concept that says you can't simultaneously know a particles exact position and momentum with perfect accuracy; the more precisely you measure one, the less precisely you can know the other. Specifically,

$$\Delta x \Delta p \geq \frac{\hbar}{2}$$

Looking at our final Simulation outcomes we can see we never cross this boundary

7 Ehrenfest's Theorem

Ehrenfest's theorem shows how quantum particles follow classical laws on average. It says that the expectation values of position and momentum obey:

$$\frac{d\langle x \rangle}{dt} = \frac{\langle p \rangle}{m}$$

which is literally the momentum equation

$$p = mv$$

Calculating each of these values and working out an error we can get a value with only $\approx 0.0757\%$ which taking into account numerical noises due to the fact we used analytical FFT it's extremely accurate.

Ehrenfest's theorem shows us that, while quantum particles have wave-like properties, their average motion mirrors classical physics.

8 Issues

Some issues do arise in the fact that numerical errors increase with higher numbers of N_{max}. This is due to the accumulation of smaller noises when evaluating the analytical FFT of multiple states. Despite the errors we can still see how our results remain consistent with quantum principles, such as the Heisenberg uncertainty principle and Ehrenfest's theorem. Addition in this model we had

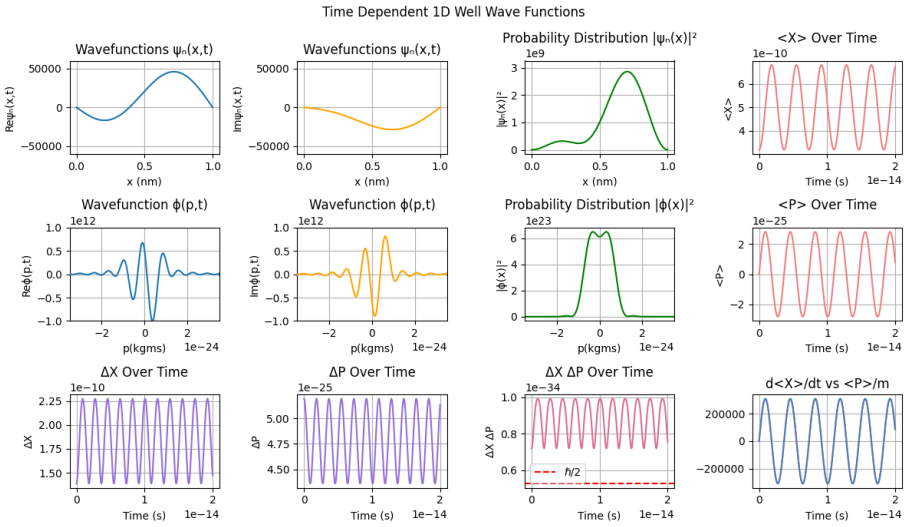

Figure 4: All Plots for a particle in a 1D potential where $N_{max} = 2$

no control over the coefficient C_n making each superposition equally likely. This approach while making the normalization easy also highlights how the probability distribution changes over time, how expectation values shift with added states, and how the interplay between multiple energy levels leads to dynamic, time-dependent behavior in a quantum system.

9 Code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import quad
from scipy import constants as sci
from matplotlib.animation import FuncAnimation

# Parameters
res = 5000
L = 1e-9
x = np.linspace(0, L, res)
dx = x[1] - x[0]
N = len(x)
t=0.0
tvalues = np.linspace(0,2e-14,res)
n_max = 2
```

```
momentumBounds = n_max * np.pi * sci.hbar / L
p = np.linspace(-12*momentumBounds, 12*momentumBounds, res)

#--------------POSITION SPACE--------------#
# Wavefunction
def psi(x, n):
    return np.sqrt(2/L) * np.sin(n*np.pi*x/L)

def psi_time(x, t, n):
    return psi(x, n) * np.exp(-1j * energy(n) * t / sci.hbar)

def psi_super(x, t):
    return (1/np.sqrt(n_max)) * sum(
        psi(x, n) * np.exp(-1j * energy(n) * t / sci.hbar)
        for n in range(1, n_max + 1)
    )

# Probability density
def psi_squared(x, t):
    return np.abs(psi_super(x, t))**2

# Energy of state n
def energy(n):
    return (n**2 * np.pi**2 * sci.hbar**2) / (2 * sci.electron_mass * L**2)

# Expected X Values
def expected_x(t):
    expected_x_val, _ = quad(lambda x: x
        *np.abs(psi_super(x,t))**2 , 0, L)
    return expected_x_val

def expected_x_squared(t):
    expected_x_squared_val, _ = quad(lambda x: (x**2)*
        np.abs(psi_super(x,t))**2 , 0, L)
    return expected_x_squared_val

#--------------MOMENTUM SPACE--------------#
def phi_n(p, n):
    k = n * np.pi / L
    prefactor = np.sqrt(2 / L) / np.sqrt(2 * np.pi * sci.hbar)
    numerator = k * (1 - (-1)**n * np.exp(-1j * p * L / sci.hbar))
    denominator = k**2 - (p / sci.hbar)**2
    return prefactor * numerator / denominator

# Superposition of states
def phi_super(p, t):
```

```
    return (1/np.sqrt(n_max)) * sum(
        phi_n(p, n) * np.exp(-1j * energy(n) * t / sci.hbar)
        for n in range(1, n_max + 1)
    )

# Probability density
def phi_squared(p, t):
    return np.abs(phi_super(p, t))**2

def expected_p(t):
    expected_p_val, _ = quad(lambda p: p*np.abs(phi_super(p,t))**2 ,
-12*momentumBounds, 12*momentumBounds)
    return expected_p_val

def expected_p_squared(t):
    expected_p_squared_val, _ = quad(lambda p: (p**2),
        *np.abs(phi_super(p,t))**2 -12*momentumBounds, 12*momentumBounds)
    return expected_p_squared_val

#-------------EXPECTATION VALUES-------------#
# Calculated Expected Values
expected_x_values = [expected_x(time) for time in tvalues]
expected_x_squared_values = [expected_x_squared(time) for time in tvalues]
delta_x_values = [np.sqrt(expected_x_squared(time)-(expected_x(time)**2)),
    for time in tvalues]

expected_p_values = [expected_p(time) for time in tvalues]
expected_p_squared_values = [expected_p_squared(time) for time in tvalues]
delta_p_values = [np.sqrt(expected_p_squared(time)-(expected_p(time)**2)),
    for time in tvalues]

heisenberg_uncertainty = ,
    ([a * b for a, b in zip(delta_x_values, delta_p_values)])
dx_dt_values = np.gradient(expected_x_values, tvalues)
dx_dt_analytic = np.array(expected_p_values) / sci.electron_mass

#---------------PLOTTING-----------------#
x_nm = x * 1e9
fig, axes = plt.subplots(3, 4, figsize=(12, 7))

(ax_psi, ax_psi_im, ax_psi_dist, ax_exp_x),
(ax_phi, ax_phi_im, ax_phi_dist, ax_exp_p),
(ax_delta_x, ax_delta_p, ax_heisenberg, ax_ehren) = axes

line_psi,= ax_psi.plot(x_nm, np.real(psi_super(x, 0)),label='Re[psi(x,t)]')
line_psi_im,= ax_psi_im.plot(x_nm, np.imag(psi_super(x, 0)),label='Im[psi(x,t)]'
```

```
line_prob, = ax_psi_dist.plot(x_nm,psi_squared(x,0),color='green')

line_phi, = ax_phi.plot(p, np.real(phi_super(p, 0)), label='Re[phi(p,t)]')
line_phi_im, = ax_phi_im.plot(p, np.imag(phi_super(p, 0)), label='Im[phi(p,t)]',
line_phi_prob, = ax_phi_dist.plot(p, phi_squared(p, 0), color='green')

# Formatting
ax_psi.set_title('Wavefunctions psi_n(x,t)')
ax_psi.set_xlabel('x (nm)')
ax_psi.set_ylabel('Re psi_n(x,t)')
ax_psi.grid(True)
ax_psi.set_ylim(-6e4, 6e4)

ax_psi_im.set_title('Wavefunctions psi_n(x,t)')
ax_psi_im.set_xlabel('x (nm)')
ax_psi_im.set_ylabel('Im psi_n(x,t)')
ax_psi_im.grid(True)
ax_psi_im.set_ylim(-6e4, 6e4)

ax_psi_dist.set_title('Probability Distribution |psi_n(x)|^2')
ax_psi_dist.set_xlabel('x (nm)')
ax_psi_dist.set_ylabel('|psi_n(x)|^2')
ax_psi_dist.grid(True)

ax_exp_x.plot(tvalues, expected_x_values, color='lightcoral')
ax_exp_x.set_title('Expected X Over Time')
ax_exp_x.set_xlabel('Time (s)')
ax_exp_x.set_ylabel('Expected X')
ax_exp_x.grid(True)

ax_delta_x.plot(tvalues, delta_x_values, color='mediumpurple')
ax_delta_x.set_title('Delta X Over Time')
ax_delta_x.set_xlabel('Time (s)')
ax_delta_x.set_ylabel('Delta X')
ax_delta_x.grid(True)

ax_phi.set_title('Wavefunction phi(p,t)')
ax_phi.set_xlabel('p (kg*m/s)')
ax_phi.set_ylabel('Re phi(p,t)')
ax_phi.grid(True)
ax_phi.set_xlim(-3.5e-24,3.5e-24)
ax_phi.set_ylim(-10e11,10e11)

ax_phi_im.set_title('Wavefunction phi(p,t)')
ax_phi_im.set_xlabel('p (kg*m/s)')
ax_phi_im.set_ylabel('Im phi(p,t)')
```

```
ax_phi_im.grid(True)
ax_phi_im.set_xlim(-3.5e-24,3.5e-24)
ax_phi_im.set_ylim(-10e11,10e11)

ax_phi_dist.set_title('Probability Distribution |phi(p)|^2')
ax_phi_dist.set_xlabel('p (kg*m/s)')
ax_phi_dist.set_ylabel('|phi(p)|^2')
ax_phi_dist.grid(True)
ax_phi_dist.set_xlim(-3.5e-24,3.5e-24)

ax_exp_p.plot(tvalues, expected_p_values, color='lightcoral')
ax_exp_p.set_title('Expected P Over Time')
ax_exp_p.set_xlabel('Time (s)')
ax_exp_p.set_ylabel('Expected P')
ax_exp_p.grid(True)

ax_delta_p.plot(tvalues, delta_p_values, color='mediumpurple')
ax_delta_p.set_title('Delta P Over Time')
ax_delta_p.set_xlabel('Time (s)')
ax_delta_p.set_ylabel('Delta P')
ax_delta_p.grid(True)

ax_heisenberg.plot(tvalues, heisenberg_uncertainty, color='palevioletred')
ax_heisenberg.set_title('Delta X * Delta P Over Time')
ax_heisenberg.set_xlabel('Time (s)')
ax_heisenberg.set_ylabel('Delta X * Delta P')
ax_heisenberg.axhline(y=sci.hbar/2, color='red', linestyle='--',
    label='hbar/2')
ax_heisenberg.grid(True)
ax_heisenberg.legend()

ax_ehren.set_title('d(Expected X)/dt vs Expected P / m')
ax_ehren.plot(tvalues, dx_dt_values, color='hotpink')
ax_ehren.plot(tvalues, dx_dt_analytic, color='steelblue')
ax_ehren.grid(True)
ax_ehren.set_xlabel('Time (s)')

# Animation function
def animate(frame):
    t = frame * 1e-16
    line_psi.set_ydata(np.real(psi_super(x, t)))
    line_psi_im.set_ydata(np.imag(psi_super(x, t)))
    line_prob.set_ydata(psi_squared(x, t))
    psi_area, psi_error = quad(lambda x: psi_squared(x, t), 0, L)
    line_phi.set_ydata(np.real(phi_super(p, t)))
    line_phi_im.set_ydata(np.imag(phi_super(p, t)))
```

```
        line_phi_prob.set_ydata(phi_squared(p, t))
        phi_area, phi_error = quad(lambda p: phi_squared(p, t), p[0], p[-1])
        diff = np.abs(dx_dt_values[frame]-dx_dt_analytic[frame])
        print(f'----------{n_max}----------')
        print(f'Time: {t}')
        print(f'Integral |psi_n(x,t)|^2 dx = {psi_area:.6f} (+- {psi_error:.2e})')
        print(f'Integral |phi(p,t)|^2 dp = {phi_area:.6f} (+- {phi_error:.2e})')
        print(f'Expected X = {expected_x(t)*1e9} nm')
        print(f'Expected X^2 = {expected_x_squared(t)}')
        print(f'Expected P = {expected_p(t)} kg*m/s')
        print(f'Expected P^2 = {expected_p_squared(t)}')
        print(f'Delta X * Delta P = {heisenberg_uncertainty[frame]}')
        print(f'Ehrenfest Check Difference = {diff}')
        print(f'% Error = {np.abs(diff) / np.abs(dx_dt_analytic[frame]) * 100}%')
        print(f'----------')
        if heisenberg_uncertainty[frame] < (sci.hbar/2):
            print('Heisenberg Uncertainty Principle Broken')

        return line_psi, line_psi_im, line_prob, line_phi, line_phi_im

anim = FuncAnimation(fig, animate, frames=200, interval=50, blit=True)
fig.suptitle('Time Dependent 1D Well Wave Functions')
plt.tight_layout()
plt.show()
```